

인공지능

Artificial Intelligence

텍스트 마이닝을 위한 딥러닝 기법

- 11. Word2Vec, ELMo, Doc2Vec의 이해
- 14. Bert의 이해와 간단한 활용
- 15. Bert 사전학습 모형에 대한 미세조정 학습
- 16. 한국어 문서에 대한 BERT활용

11장 Word2Vec, ELMo, Doc2Vec의 이해

- 이번 장은 워드 임베딩의 대표적인 기법인 Word2Vec과 ELMo, 그리고 Doc2Vec을 다룸
- Doc2Vec은 보기 드물게 워드가 아닌 문서를 직접 임베딩함
- Word2Vec과 ELMo는 사전학습을 통해 워드 임베딩을 수행하고, 전이학습을 통해 워드 임베딩 결과를 공유한다는 점에서 일반적인 학습에서의 워드 임베딩과 차이가 있음

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- Word2Vec 이란?

- Word2Vec은 가장 널리 알려진 워드 임베딩 기법임
- 단어를 원핫 벡터로 표현하는 것은 기계적이지만, 이 원핫 벡터를 밀집 벡터로 변환하려면 반드시 학습이 필요
- Word2Vec 기법에서 학습의 목적은 단어에 의미적인 정보를 함축함으로써 유사도를 계산하거나 연산을 수행하고, 더 나아가 학습된 결과를 다른 작업에서도 사용할 수 있는 전이학습을 지원하는 것

- Word2Vec 학습의 원리

- 사람이 언어에 대해서 이해 한다는 것은 문장에 사용되는 단어에 대한 예측이 가능하다는 것이며, 이러한 예측은 단어들의 순서, 즉 문맥을 이해할 때에만 가능함
- Word2Vec은 다음 단어 혹은 주변 단어에 대한 예측을 잘할 수 있도록 학습함으로써 문맥을 이해시키고, 밀집 벡터에 그러한 문맥 정보를 담으려는 시도라고 이해할 수 있음

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- Word2Vec 학습의 원리

- Word2Vec은 CBOW와 Skip-Gram의 두 가지 학습 방식을 갖고 있음
- (1) CBOW는 주변의 단어를 이용해 중심 단어를 예측하도록 학습을 수행함
- (2) Skip-Gram은 중심의 한 단어를 이용해 주변의 단어를 예측함

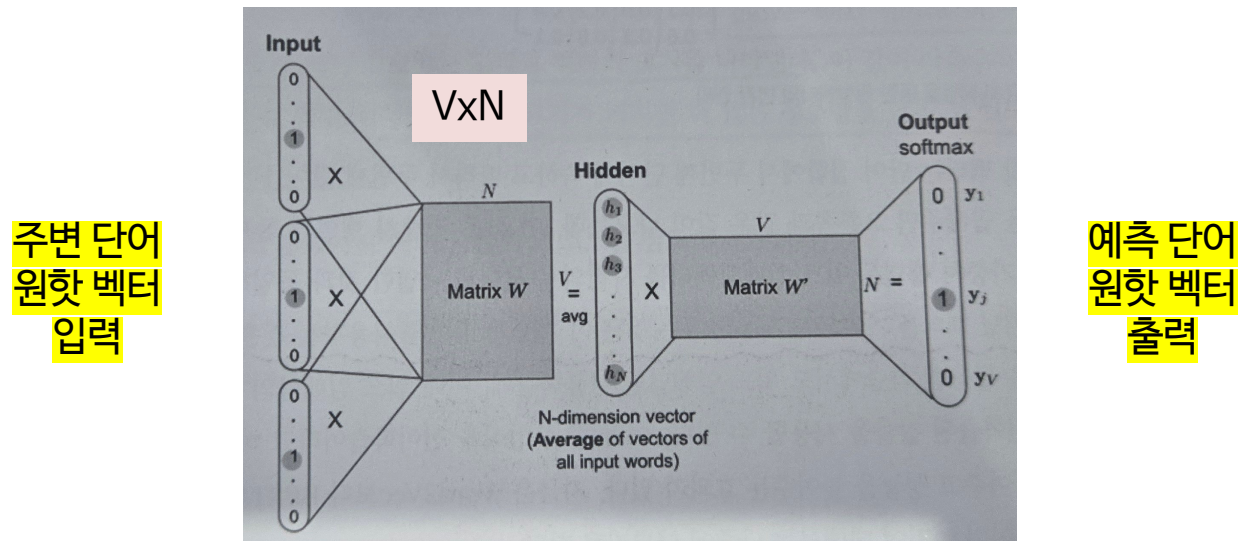
- CBOW 학습 방법

- “Alice was beginning to get very tired of sitting by her sister on the bank” 라는 문장이 있을 때, 앞에 있는 “Alice”, “was”의 두 단어와 뒤에 있는 “to”, “get”으로 “beginning”을 예측하도록 학습함
- 이때 앞뒤 단어들을 몇 개씩 예측에 사용할지 결정하는 범위를 윈도우(window)라고 함
- 위의 예시는 각 두 단어씩을 사용하므로 윈도우 크기는 2임
- 만일 윈도우 크기가 10이라면, “was”와 “to”만으로 예측해야 함
- 윈도우를 앞에서부터 한 칸 이동하면 다음에는 “was”, “beginning”과 “get”, “very”로 “to”를 예측함
- 즉, 문장에서 윈도우를 뒤로 이동하면 계속해서 학습에 사용할 입력과 출력을 만들 수 있음

11.1 Word2Vec - 대표적인 워드 임베딩 기법

▪ CBOW 학습 방법

- 아래의 그림은 CBOW의 구조를 가장 잘 설명해주는 그림임
- 세 개의 주변 단어를 이용해 하나의 중심 단어를 예측하고 있음
- 세 개의 단어는 모두 원핫 벡터로 먼저 표현되며, 벡터의 크기는 사용된 어휘의 수와 동일하고 V 로 표현
- 은닉 노드의 크기가 N 인 이유는 “세 개의 밀집 벡터에 대해 평균”을 구해서 하나의 벡터로 만들었기 때문
- 가중치행렬 W 는 V 크기의 원핫 벡터를 N 크기의 밀집 벡터로 변환해야 하므로 크기는 $V \times N$ 임
- 가중치행렬 W' 는 N 크기의 은닉 벡터를 V 크기의 출력 벡터로 변환해야 하므로 크기는 $N \times V$ 임



11.1 Word2Vec - 대표적인 워드 임베딩 기법

▪ CBOW 학습 방법

- 가중치행렬 W 와 W' 각각은 학습 대상이며, W 의 한 행이 각 단어에 대한 밀집 벡터에 해당 함
- 사전학습된 워드 임베딩 벡터를 가져와서 사용한다는 의미는 W 를 사용하는 것을 의미함
- 사전학습된 값을 사용하는 방식을 전이학습이라고 하며, 필요한 학습의 양을 줄여주고 성능을 높여주는 효과가 있음
- 최근에는 Word2Vec보다 BERT의 사전학습 모형이 많이 쓰임

$$[0 \ 1 \ 0 \ 0 \ 0] \times \begin{bmatrix} 0.2 & 0.9 & 0.5 & 0.3 \\ 0.7 & 0.4 & 0.2 & 0.8 \\ 0.4 & 0.1 & 0.5 & 0.7 \\ 0.9 & 0.8 & 0.4 & 0.6 \\ 0.1 & 0.9 & 0.2 & 0.4 \\ 0.6 & 0.2 & 0.8 & 0.1 \end{bmatrix} = [0.7 \ 0.4 \ 0.2 \ 0.8]$$

가중치 행렬을 통한 원핫벡터의 변환 원리

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- skip-gram 학습 방법
 - CBOW와 달리 중심 단어를 입력으로 하여 주변의 여러 단어를 예측하는 방법
 - Word2Vec의 학습에서는 Noise Contrastive Estimation, Negative Sampling과 같은 복잡한 아이디어가 사용됐는데, Skip-gram과 학습 방식에 대한 상세한 설명은 이 책의 범위를 넘어가므로 생략함

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- Word2Vec 활용 - 학습된 모형 가져오기

- Word2Vec을 활용하는 방법

- 1) 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (from scratch)
 - 2) 사전학습된 임베딩 벡터를 가져다 활용하는 법 (from pretrained model)

- 사전학습된 임베딩 벡터를 활용하는 법

- 대표적인 라이브러리 Gensim 패키지를 사용

- <https://radimrehurek.com/gensim/models/word2vec.html>

- 제공하는 사전학습된 모델

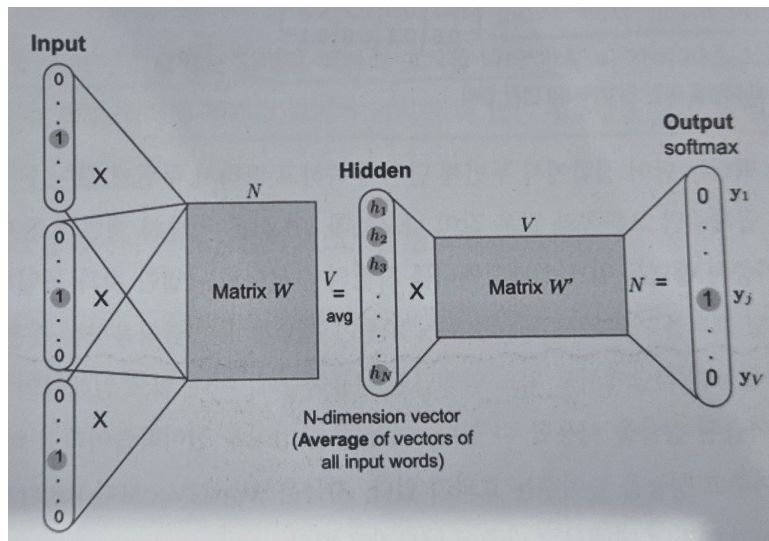
- Gensim 이 제공하는 사전학습 모델들은 대부분 외부 연구기관에서 공개한 벡터를 gensim 포맷만 맞춰서 제공하는 형태임

```
>>> import gensim.downloader
>>> # Show all available models in gensim-data
>>> print(list(gensim.downloader.info()['models'].keys()))
['fasttext-wiki-news-subwords-300',
 'conceptnet-numberbatch-17-06-300',
 'word2vec-ruscorpora-300',
 'word2vec-google-news-300',
 'glove-wiki-gigaword-50',
 'glove-wiki-gigaword-100',
 'glove-wiki-gigaword-200',
 'glove-wiki-gigaword-300',
 'glove-twitter-25',
 'glove-twitter-50',
 'glove-twitter-100',
 'glove-twitter-200',
 '__testing_word2vec-matrix-synopsis']
>>>
```

- Ruscorpora : 러시아국립코퍼스 기반으로 학습된 러시아어 모델
- Google-news : 구글 뉴스 데이터 학습 모델

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 사전학습된 임베딩 벡터를 활용하는 법
 - “word2vec-google-news-300” 을 분석해보자.
 - 구글 뉴스 데이터의 어휘 수 (주요 단어 수, vocabulary size)는 300만개
 - 사전학습 모델 이름에 있는 300차원의 의미는 임베딩 벡터의 차원
 - 입출력 원핫 벡터의 길이(차원)는 3,000,000
 - 은닉 벡터의 길이(차원)는 300



사전학습 입력 가중치 W
= 300만 단어 x 300차원
= 약 9억 floating values
= 약 3.6 GB

참고로, 출력 가중치 W' 은 공개하지 않고
사전학습 가중치 다운로드 시 W 만 제공

11.1 Word2Vec - 대표적인 워드 임베딩 기법

■ 사전학습된 임베딩 벡터를 활용하는 법 (gensim)

```
In [1]: # https://wikidocs.net/233744
# 자연어 처리 라이브러리 : gensim - 단어 단위임베딩 모델

import gensim
from gensim.models import Word2Vec
from gensim.models import KeyedVectors

# binary=True 옵션 필수 (Google News 모델은 binary format)
model_path = "/kaggle/input/word2vec/pytorch/default/1/GoogleNews-vectors-negative300.bin"
word2vec = KeyedVectors.load_word2vec_format(model_path, binary=True)

In [2]: # 단어 vector 얻기 - 300차원의 벡터
vector = word2vec["king"]

# 단어 간 유사도
similarity = word2vec.similarity("king", "queen")

# 가장 유사한 단어
result = word2vec.most_similar("computer", topn=5)

In [3]: vector.shape

Out[3]: (300, )

In [4]: similarity

Out[4]: 0.6510956

In [5]: print(result)

[('computers', 0.7979379892349243), ('laptop', 0.6640493273735046), ('laptop_computer', 0.6548
868417739868), ('Computer', 0.647333562374115), ('com_puter', 0.608208006599426)]
```

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (gensim)
 - 구글 뉴스 사전학습 모델로 IMDB 직접 학습하기

```
In [11]: import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from gensim.models import Word2Vec

#####
# 1. 환경 설정
#####
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

#####
# 2. IMDB 데이터 로드
#####
df = pd.read_csv('/kaggle/input/imdb-dataset-of-50k-movie-reviews/IMDB Dataset.csv')

# train/test 분리
train_df, test_df = train_test_split(df, test_size=0.5, random_state=42)
train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print("Train size:", len(train_df))
print("Test size:", len(test_df))
print(train_df.head())

Device: cuda
Train size: 25000
Test size: 25000

          review sentiment
0  "Congo" is based on the best-selling novel by ... negative
1  Wow, here it finally is; the action "movie" wi... negative
2  'Identity . . . . I am part of my surrounding... positive
3  "Sir" John Gielgud must have become senile to ... negative
4  Below average movie with poor music considerin... negative
```

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (gensim)
 - 구글 뉴스 사전학습 모델로 IMDB 직접 학습하기

```
In [7]: #####
# 3. 토큰나이저 정의
#####
def tokenizer(text):
    return text.lower().split()

#####
# 4. gensim Word2Vec 학습
# - 매뉴얼 : https://radimrehurek.com/gensim/models/word2vec.html
#####

# Word2Vec 학습데이터 준비
sentences = [tokenizer(text) for text in train_df["review"]]

w2v_dim = 100 # 벡터 차원
w2v_model = Word2Vec(
    sentences=sentences,
    vector_size=w2v_dim,
    window=5,
    min_count=3, # 더 큰 min_count = 노이즈 감소 but vocab 작아짐
    workers=4,
    sg=1 # skip-gram (1), CBOW (0)
)

# 메모리 최적화 -> KeyedVectors만 유지
w2v_model = w2v_model.wv

print("Word2Vec vocab size:", len(w2v_model))
```

Word2Vec vocab size: 68824

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (gensim)
 - 구글 뉴스 사전학습 모델로 IMDB 직접 학습하기

```
In [9]: #####
# 6. Dataset 정의
#####
class IMDBW2VDataset(Dataset):
    def __init__(self, dataframe, w2v_model):
        self.df = dataframe
        self.w2v = w2v_model
        self.dim = w2v_model.vector_size

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        text = self.df.loc[idx, "review"]
        sentiment = self.df.loc[idx, "sentiment"]

        tokens = tokenizer(text)
        vec = sentence_to_vec(tokens, self.w2v, self.dim) # 평균 벡터임

        x = torch.tensor(vec, dtype=torch.float32)
        y = torch.tensor(1.0 if sentiment == "positive" else 0.0, dtype=torch.float32)

        return x, y

# 즉, train_dataset을 단어 단위 임베딩으로 어떻게 만들었는지 봐야함
train_dataset = IMDBW2VDataset(train_df, w2v_model)
test_dataset = IMDBW2VDataset(test_df, w2v_model)

batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
In [8]: #####
# 5. 문장을 평균 Word2Vec 벡터로 변환
#####
def sentence_to_vec(tokens, w2v, dim):
    vecs = []
    for w in tokens:
        if w in w2v:
            vecs.append(w2v[w])
    if len(vecs) == 0:
        return np.zeros(dim, dtype=np.float32)

    return np.mean(np.stack(vecs, axis=0), axis=0).astype(np.float32)
```

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (gensim)
 - 구글 뉴스 사전학습 모델로 IMDB 직접 학습하기

```
In [12]: #####
# 7. MLP 모델 정의
#####
class W2V_MLP(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 1) # Binary classification logits
        )

    def forward(self, x):
        return self.net(x).squeeze(1)

model = W2V_MLP(w2v_dim).to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

```
In [13]: #####
# 8. 학습 & 평가 함수
#####
def train_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, total_correct, total = 0.0, 0, 0

    for x, y in loader:
        x, y = x.to(device), y.to(device)

        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)

        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)
        preds = (torch.sigmoid(logits) >= 0.5).float()
        total_correct += (preds == y).sum().item()
        total += y.size(0)

    return total_loss / total, total_correct / total

def eval_epoch(model, loader, criterion):
    model.eval()
    total_loss, total_correct, total = 0.0, 0, 0

    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            logits = model(x)
            loss = criterion(logits, y)

            total_loss += loss.item() * x.size(0)
            preds = (torch.sigmoid(logits) >= 0.5).float()
            total_correct += (preds == y).sum().item()
            total += y.size(0)

    return total_loss / total, total_correct / total
```

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (gensim)
 - 구글 뉴스 사전학습 모델로 IMDB 직접 학습하기

```
In [14]: #####
# 9. 학습 루프
#####
num_epochs = 5
for epoch in range(1, num_epochs + 1):
    train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer)
    test_loss, test_acc = eval_epoch(model, test_loader, criterion)

    print(f"[Epoch {epoch}] "
          f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
          f"Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.4f}")

[Epoch 1] Train Loss: 0.6022, Train Acc: 0.6936 | Test Loss: 0.4875, Test Acc: 0.8054
[Epoch 2] Train Loss: 0.4386, Train Acc: 0.8124 | Test Loss: 0.4052, Test Acc: 0.8243
[Epoch 3] Train Loss: 0.3965, Train Acc: 0.8292 | Test Loss: 0.3797, Test Acc: 0.8361
[Epoch 4] Train Loss: 0.3829, Train Acc: 0.8347 | Test Loss: 0.3731, Test Acc: 0.8383
[Epoch 5] Train Loss: 0.3772, Train Acc: 0.8373 | Test Loss: 0.3661, Test Acc: 0.8405
```


11.1 Word2Vec - 대표적인 워드 임베딩 기법

- Word2Vec이 워드 임베딩의 가능성을 잘 보여주며 많은 인기를 얻으며, 가진 단점을 보완하기 위한 다양한 워드 임베딩 방법들이 제안 되었음 (Glove, FastText)
- FastText 이란?
 - Facebook에서 개발되었음 (2016, FAIR)
 - Word2Vec의 또 다른 문제는 학습 문서에 없는 단어에 대해 추약하다는 것
 - 학습된 적이 없는 단어를 만나게 되면 대부분의 경우 OOV(out of vocabulary) 토큰을 부여하는 방식으로 처리되며, 글자 그대로 '모르는 단어' (UNK, unknown vector)이므로 다른 단어와의 유사도를 계산하는 것이 불가능함
 - 이를 해결하기 위해 FastText는 단어에 문자 단위의 N-gram을 적용함으로써 이를 해결하고자 함

```
예:
css
["cat", "dog", "table", "apple", "<UNK>"]

입력 문장:
arduino
"The AI robot walks."

Word2Vec vocab에 없음 → 모두 <UNK> 로 변환:
php-template
"<UNK> <UNK> walks"
```

11.1 Word2Vec - 대표적인 워드 임베딩 기법

▪ FastText 학습 원리

- 통상 N-gram 이라고 하면 단어 단위로 이루어지는 것을 말함
- 예를 들어, “It is nice to meet you” 라는 문장에 대해 N=2인 bi-gram을 적용하면, “it is”, “is nice”와 같은 토큰들을 만들어 냄
- FastText는 이와 같은 방식을 문자 단위로 적용함
- 예를 들어, “Hello” 라는 단어에 대해 N=2인 bi-gram을 적용하면, “He”, “el”, “ll”, “lo”와 같은 토큰들을 만들어 냄
- FastText에서는 문자 단위 토큰을 부분단어(Subword)라고 부름
- 단어의 일부를 토큰으로 사용함으로써 모르는 단어(OOV)에 대해서도 유사도를 계산하는 것이 가능해짐
- 기존의 단어를 변형하거나 결합해서 만든 새로운 단어에 대해서도 뛰어난 유사도 계산을 할 수 있음

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- FastText 활용

- FastText을 활용하는 방법

- 1) 대상 말뭉치를 직접 학습하여 나만의 워드 임베딩 벡터를 생성하는 법 (from scratch)
 - 2) 사전학습된 임베딩 벡터를 가져다 활용하는 법 (from pretrained model)

- 사전학습된 임베딩 벡터를 활용하는 법

- 대표적인 라이브러리 Gensim 패키지를 사용

- <https://radimrehurek.com/gensim/models/fasttext.html>

- 제공하는 사전학습된 모델

- Gensim 이 제공하는 사전학습 모델들은 대부분 외부 연구기관에서 공개한 벡터를 gensim 포맷만 맞춰서 제공하는 형태임

```
>>> import gensim.downloader
>>> # Show all available models in gensim-data
>>> print(list(gensim.downloader.info()['models'].keys()))
['fasttext-wiki-news-subwords-300',
 'conceptnet-numberbatch-17-06-300',
 'word2vec-ruscorpora-300',
 'word2vec-google-news-300',
 'glove-wiki-gigaword-50',
 'glove-wiki-gigaword-100',
 'glove-wiki-gigaword-200',
 'glove-wiki-gigaword-300',
 'glove-twitter-25',
 'glove-twitter-50',
 'glove-twitter-100',
 'glove-twitter-200',
 '__testing_word2vec-matrix-synopsis']
>>>
```

- Wiki - 위키피디아 전체 + 뉴스 관련 문서

- 위키피디아 2017 : 약 30억 단어 규모(3B tokens)

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 사전학습된 임베딩 벡터를 활용하는 법

- “fasttext-wiki-news-subwords-300” 을 분석해보자.
 - 데이터의 어휘 수 (주요 단어 수, vocabulary size)는 100만개 (1M)
 - Subword n-gram vocabulary 는 2M~3M
 - 사전학습 모델 이름에 있는 300차원의 의미는 단어 벡터의 차원을 의미함
 - 단어 벡터가 300이면, subword embedding도 300 차원이어야 하며, 이들의 평균을 취함
 - 즉, fasttext는 2M을 300차원으로 임베딩하는 가중치 학습

[Word2Vec]

단어 원형 → **W_{in}** → 단어벡터(300) → **W_{out}** → softmax/neg-sampling

[fastText]

단어 → subword 분해 → (각 subword embedding) → 합산 = 단어벡터(300) → **W_{out}** → softmax/neg-sampling



11.1 Word2Vec - 대표적인 워드 임베딩 기법

■ 사전학습된 임베딩 벡터를 활용하는 법 (gensim) - fasttext

1. fasttext 모델

- 사전학습모델 다운로드 <https://fasttext.cc/docs/en/english-vectors.html>

```
from gensim.models import KeyedVectors

# fasttext-wiki-news-subwords-300 다운로드 및 로드
model = KeyedVectors.load_word2vec_format(
    '/kaggle/input/wiki-news-300d-1m-subword/pytorch/default/1/wiki-news-300d-1M-subword.vec',
    binary=False
)
```

```
# 300차원 단어 벡터 가져오기
vec = model['king']
vec.shape
```

[22... (300,)]

```
# 단어 간 유사도
similarity = model.similarity("run", "running")
print(similarity)
```

0.8197942

```
# 단어 간 유사도
similarity = model.similarity("big", "bigger")
print(similarity)
```

0.78803325

+ Code + Markdown

11.1 Word2Vec - 대표적인 워드 임베딩 기법

- 사전학습된 임베딩 벡터를 활용하는 법 (gensim) - fasttext

2. word2vec 모델

```
[25]: model_path = "/kaggle/input/googlenews-vectors-300/pytorch/default/1/GoogleNews-vectors-negative300.bin"
word2vec = KeyedVectors.load_word2vec_format(model_path, binary=True)
```

```
[26]: # 단어 간 유사도
similarity = word2vec.similarity("run", "running")
print(similarity)

0.60629654
```

```
[27]: # 단어 간 유사도
similarity = word2vec.similarity("big", "bigger")
print(similarity)

0.68423855
```

11.2 ELMo - 문맥에 따른 단어 의미의 구분

▪ Word2Vec의 문제점

- Word2Vec은 단어를 밀집 벡터로 임베딩해서 활용하는 다양한 방법을 보여주었음
- 대형 말뭉치를 대상으로 학습한 밀집 벡터를 가져와서 활용하는 전이학습의 활용 가능성도 보여주었음
- word2vec의 가장 큰 단점은 동음이의어 처리임
- 우리말 ‘배’가 먹는 배, 타는 배, 가슴 밑에 있는 배 등 다양한 의미가 있듯이 영어도 동일한 단어가 여러 의미를 갖는 경우가 많음. 그러나 word2vec에서는 모든 ‘배’가 동일한 벡터로 임베딩됨
- ‘배’의 의미에 따라 서로 다른 벡터로 임베딩할 수 있다면, 우리가 원래 추구하던 문맥 파악을 더 잘할 수 있을 것이고, 자연어 처리의 성능도 향상될 수 있을 것임

▪ ELMo - 문맥을 반영한 워드 임베딩

- ELMo (Embeddings from Language Model)는 이를 위해 만들어진 문맥을 반영한 워드 임베딩 모형임
- Word2vec은 임베딩 벡터(값)가 고정이지만, ELMo는 가변적인 임베딩 벡터(값)를 생성함

11.2 ELMo - 문맥에 따른 단어 의미의 구분

- ELMo의 구조

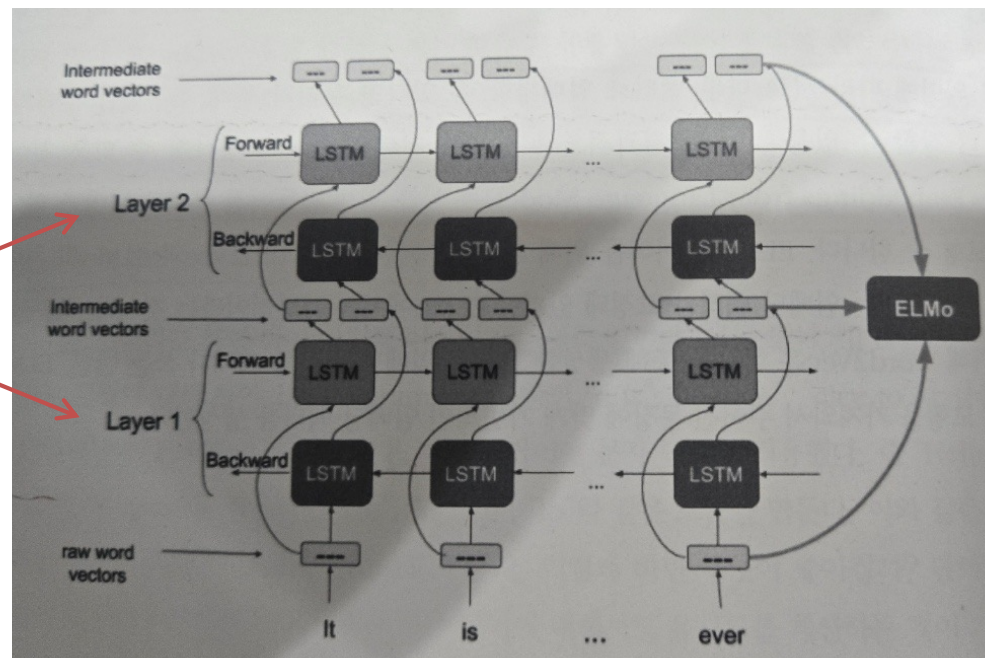
- ELMo는 언어 모델(Language Model)을 이용해 임베딩을 수행함
- 일반적으로 언어 모델은 앞에 나온 단어들을 이용해 다음 단어를 예측하는 모형임
- 예를 들어, '나는 배가 고파서 밥을 먹었다.'라는 문장에서, '나는 배가 고파서 밥을'까지만 들어도 그 다음 단어가 '먹었다'가 될 수 있을 것으로 예측할 수 있음
- 이와 같은 예측은 문맥에 대한 이해가 있어야 가능하며, 학습을 통해 다음 단어를 잘 예측하도록 모델을 만들면 문맥에 대한 이해가 높아졌다고 볼 수 있음

11.2 ELMo - 문맥에 따른 단어 의미의 구분

▪ ELMo의 구조

- ELMo는 사전학습된 양방향 LSTM(bi-LSTM)을 사용해 임베딩을 수행함
- 양방향 LSTM은 앞 단어들이 뒤에 미치는 영향에 더해, 뒤 단어에서 앞 단어 방향으로도 LSTM 층을 추가해서 반대 방향의 영향도 학습하는 특징이 있음
- 우선 이 모형은 아래 그림과 같이 두 개의 양방향 LSTM 층으로 이루어져 있음

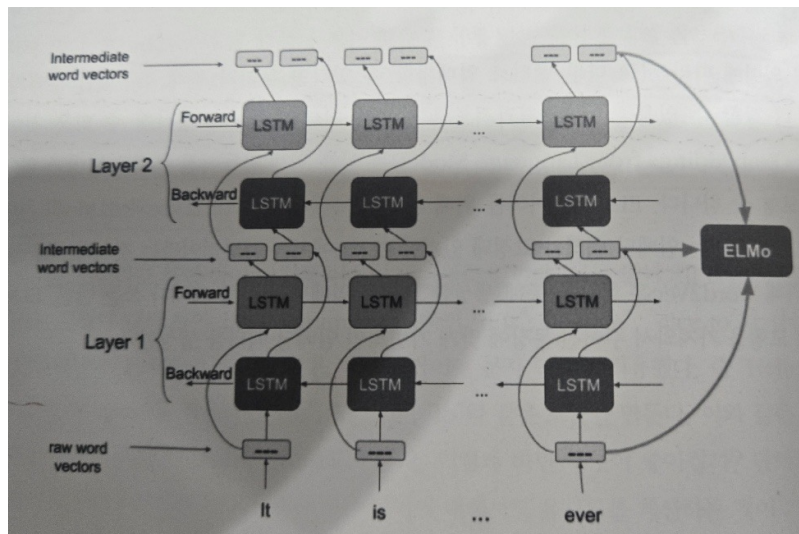
두 개의
양방향 LSTM



11.2 ELMo - 문맥에 따른 단어 의미의 구분

▪ ELMo의 구조

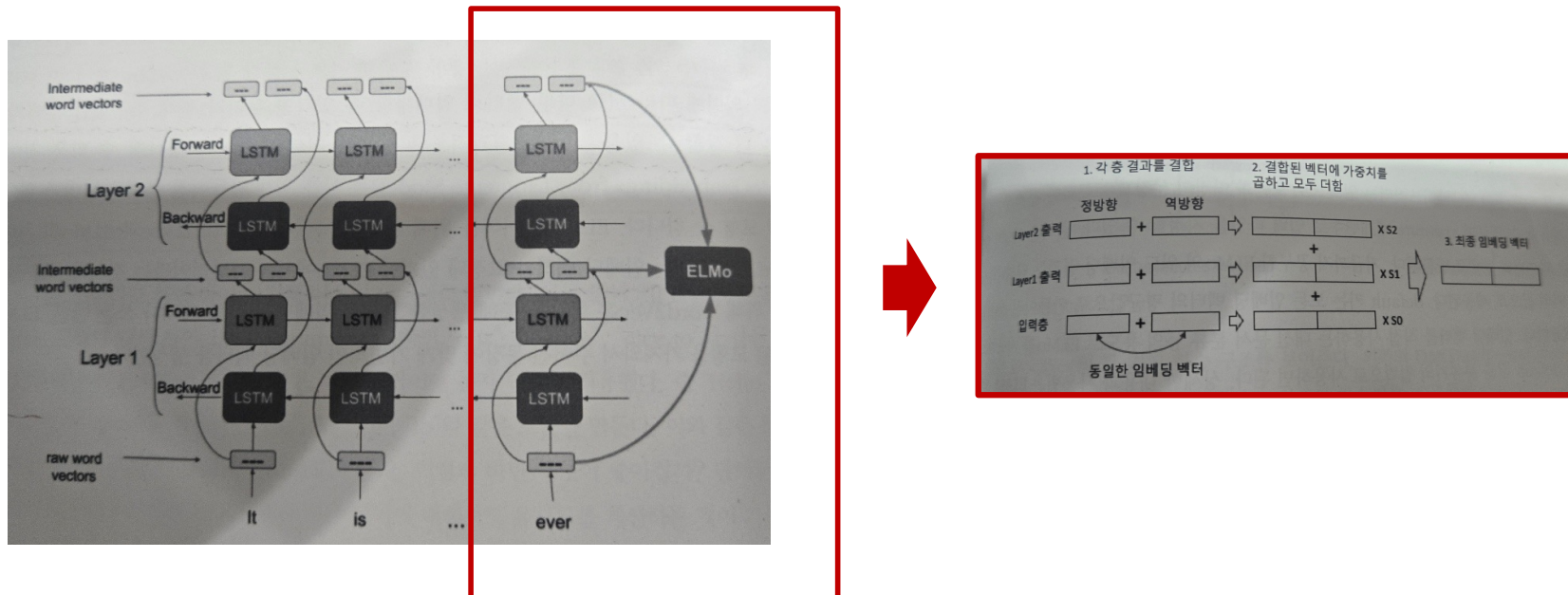
- 문장의 단어들은 일반적인 임베딩을 통해 벡터로 변환되고, 이 벡터가 첫째 양방향 LSTM층의 입력이 됨
- 동일한 임베딩 벡터가 정방향 LSTM과 역방향 LSTM에 입력으로 들어가 각각의 출력을 만들고, 이 출력은 합쳐져서 첫번째 층의 출력이 됨
- 단, 첫번째 층의 출력은 두번째 층의 입력으로 들어갈 때, 합쳐지지 않고 각각 입력으로 사용됨
- 정방향은 계속 정방향으로만, 역방향은 계속 역방향으로만 층이 쌓임. 즉, 정방향 LSTM과 역방향 LSTM이 각각 독립적으로 학습된다고 할 수 있음



11.2 ELMo - 문맥에 따른 단어 의미의 구분

▪ ELMo의 구조

- 최종 ELMo 임베딩 벡터를 만드는 과정은 아래 그림과 같음
- 각 층의 정방향 LSTM과 역방향 LSTM의 결과를 이어 붙여서 하나의 벡터로 결합
- 입력층은 하나의 임베딩 벡터이므로 같은 벡터 두 개를 결합
- 두번째 단계에서는 각 층의 벡터에 가중치를 곱함. 각 층의 결과 중에서 어떤 것이 최종결과에 더 많은 영향을 미치도록 할지를 결정하는 것으로 학습을 통해 조정됨
- 마지막으로 입력층을 포함한 **모든 층의 결과를 더해서 최종 임베딩 벡터를 만듦**



11.2 ELMo - 문맥에 따른 단어 의미의 구분

- ELMo의 구조

- ELMo는 임베딩된 벡터를 전이해서 사용하는 것이 아니라 모형 자체를 전이하고 임베딩 벡터는 주어진 문장을 모형에 적용시켜서 생성함
- 따라서 대상으로 하는 문장에 따라 임베딩 벡터는 가변적인 값을 갖게 됨
- 요즘은 BERT에 밀려 활용도가 떨어지는 편임

11.3 Doc2Vec - 문맥을 고려한 문서 임베딩

- 딥러닝을 활용하는 자연어 처리 방법 대부분이 단어에 대한 임베딩을 하는 반면, Doc2Vec은 문서에 대해 직접 임베딩한다는 점에서 큰 차이가 있음
- BOW방식의 카운트 벡터 혹은 TF-IDF 벡터가 문서를 임베딩한다는 점에서 Doc2Vec과 유사한 면이 있으나, 이들과의 차이점은 단어의 순서를 고려한 문맥 정보가 BOW에서는 무시되지만 Doc2Vec에서는 임베딩 벡터에 포함된다는 점임
- Doc2Vec은 문서를 임베딩한다는 점에서 신선한 시도였으며, 지금도 단어와 문서를 함께 임베딩해야 할 때 사용됨

11.3 Doc2Vec - 문맥을 고려한 문서 임베딩

▪ Doc2Vec 원리

- Word2Vec을 이해하고 있으면, Doc2Vec 의 원리도 어렵지 않게 이해할 수 있음
- Doc2Vec에서는 문서의 ID를 단어와 동일하게 취급해서 학습과정에 포함시킴

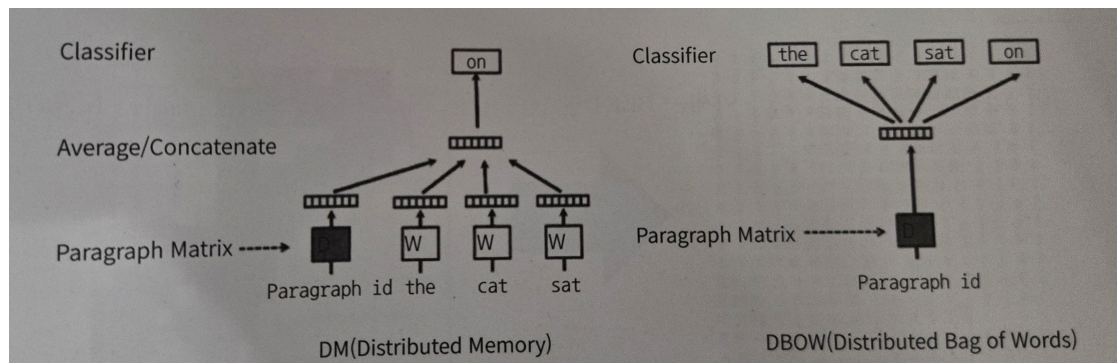
▪ DM (Distributed Memory) 구조

- Word2Vec의 CBOW에 문서 ID를 추가한 형태의 학습
- 앞의 단어들과 문서 ID를 이용해 다음 단어를 예측하는 방식으로 학습이 이루어짐

▪ DBOW (Distributed Bag of Words) 구조

*Skip-gram이란 중심 단어를 입력으로 하여 주변의 여러 단어를 예측하는 방법

- Skip-gram에 문서 ID를 추가한 형태
- 문서 ID로 일련의 단어들을 예측하는 방식으로 학습



14. BERT의 이해와 간단한 활용

- BERT(2018)는 Bidirectional Encoder Representations from Transformer 의 약자
- BERT는 현재 시점에서 텍스트 마이닝 딥러닝 모형 중 가장 중요한 모형임
- 텍스트 마이닝의 거의 전 분야에서 우수한 성능을 보이고 있기 때문임
- BERT는 처음 발표된 모형 외에 파생된 다양한 모형이 존재함
- BERT는 우수한 성능을 내기 위해 언어 모델에 기반한 고유의 학습 방법이 있음
- BERT에 대해 완전히 이해하려면 언어 모델과 학습법 그리고 BERT 구조에 대해 이해해야 함

14.1 왜 언어 모델이 중요한가?

- 언어 모델 (=언어 모형)은 문장 혹은 단어의 시퀀스에 대해 확률을 할당하는 모델을 말함
- 예를 들어, 언어 모델은 “나는 배가 고파서 밥을 먹었다.”와 “나는 배가 고파서 밥을 치웠다.” 중 더 자연스러운 문장에 더 높은 확률을 부여함
- “자연스러운”의 정의는 일반적으로 더 많이 사용되는 이라는 의미를 가지며, 사람에게 가깝게 더욱 자연스러운 문장을 알아낼 수 있는 모형이라고 할 수 있음
- 언어 모델을 이용한 학습이 갖는 의미는 더 자연스러운 문장을 알 수 있고, 이는 언어를 더 잘 이해한다는 것을 의미함
- 언어 모델의 중요한 장점은 비지도 학습이 가능하다는 것! 언어 모델은 주어진 단어의 시퀀스 뒤에 오는 단어를 예측하는 방식으로 학습하므로 문장에 대한 답을 별도로 필요로 하지 않고, 그냥 문서를 적절하게 잘라서 학습이 가능함
- **언어 모델로 언어에 대한 이해를 높인 후, 미세조정학습을 통해, 언어 모델로 학습한 모형을 문서 분류와 같은 지도학습에 활용할 수 있음

14.2 BERT의 구조

- BERT가 소개된 논문의 제목에서 알 수 있듯이, BERT는 언어 모델 기반의 학습을 이용해 언어에 대한 이해를 높이는데 목적이 있음
 - “BERT: Pre-training of Deep Bidirectional Transformer for Language Understanding”
- 물론 그 이해를 바탕으로 다양한 작업(down stream task)을 수행하는 것이 최종 목적임
- BERT 는 Bidirectional Encoder Representations from Transformers의 약자이며, **트랜스포머에서 인코더 부분만 사용한 모형으로, 이 인코더의 특징은 양방향 셀프 어텐션을 구현하고 있음.**
- BERT는 이름을 통해 왜 이런 구조를 강조하고 있을까? 그 이유는 GPT (Generative Pre-trained Transformer)와 비교해 설명하고자 했기 때문임
- GPT(2018)는 BERT(2018)보다 먼저 발표된 모형으로, 주로 자연어 문장 생성에 특화된 모델이며, **트랜스포머의 디코더 부분만 따로 떼어서 학습 모형으로 사용했음.** 즉, 인코더에서 디코더로의 어텐션은 생략돼 있고, **셀프 어텐션은 순방향만 적용됨.**
- 반면, BERT는 트랜스포머의 인코더 부분을 떼어서 사용했으며, **양방향 셀프 어텐션을 모두 활용할 수 있었음**

14.3 언어 모델을 이용한 사전학습과 미세 조정학습

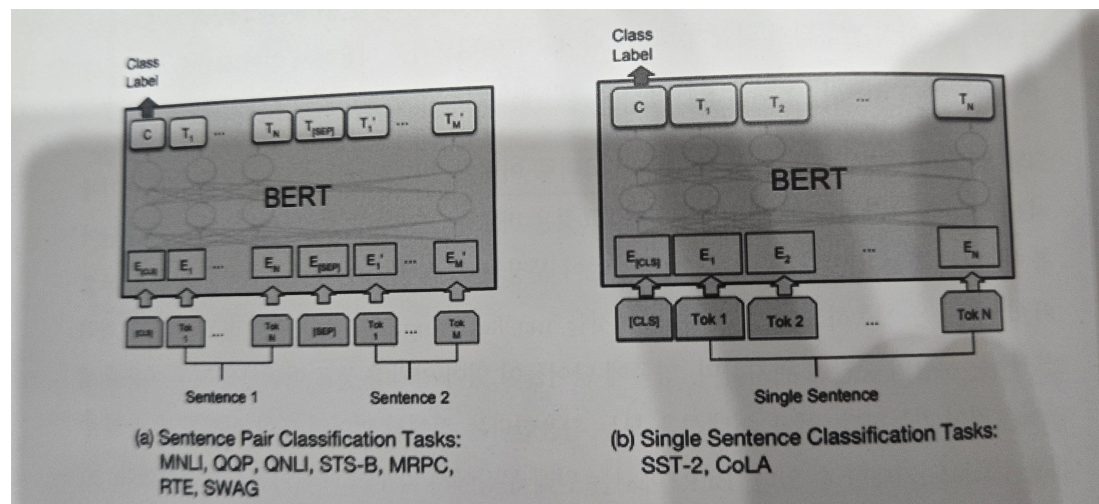
- BERT의 학습은 1)사전학습과 2)미세조정학습의 두 단계로 나뉨
 - 사전학습은 언어에 대한 이해를 높이기 위한 비지도학습을 말함
 - 미세조정학습은 실제 수행하고자 하는 작업에 대한 지도학습임
- BERT의 사전학습
 - BERT의 사전학습은 지금까지 배운 언어 모델 학습과 차이가 있음
 - 기존 언어 모델을 이용한 학습에서는 앞에 주어진 단어들 다음에 나올 단어의 확률을 예측하는 방식으로 학습하며, 이와 같은 방식은 트랜스포머의 디코더와 구조가 일치함
 - 디코더의 셀프 어텐션에는 masked 라는 용어가 들어가고 이것은 어텐션이 순방향으로만 이루어져 있음을 의미함. 현재까지의 단어 시퀀스를 이용해서 다음 단어를 예측하는 작업은, 그 이후 단어들로부터 오는 어텐션을 활용할 수 없는데, 너무나 당연하게도 그 단어들은 아직 생성되지 않았기 때문임. GPT가 디코더를 이용하여 언어 모델을 학습한 이유도 동일함

14.3 언어 모델을 이용한 사전학습과 미세 조정학습

- BERT의 사전학습
 - BERT는 양방향 셀프 어텐션 인코더를 이용해 언어 모델을 학습하였음
 - (1) 마스크를 이용한 언어 모델 학습
 - BERT만의 아이디어 masking
 - Masking이란 단어를 가리고, 가린 단어를 예측하게 하는 것. 순서에 관계 없이 문장 안에서 랜덤한 위치의 단어를 지우고, 모형이 이 단어를 예측하도록 한 것
 - BERT는 원래 문장에서 약 15%의 단어들을 마스킹함
 - (2) 두 문장을 이용한 학습
 - BERT만의 아이디어 두 문장에 대한 순서를 맞추는 것
 - A와B 두 문장을 주고 순서가 올바른지 거꾸로인지를 예측하도록 함

14.3 언어 모델을 이용한 사전학습과 미세 조정학습

- BERT의 미세조정
 - 미세조정학습은 실제로 주어진 문제를 해결하는 것
 - 대표적인 예) 하나의 문장 분류 문제, 두 문장을 입력으로 하는 분류 문제 (둘 간의 관계 분류)
 - 미세조정학습을 수행하면, 사전학습으로 미리 만들어진 가중치들이 목표에 맞게 세밀하게 조정됨
 - 특히, 분류기 토큰[CLS]가 분류 목적에 맞게 학습되고, 단어들의 어텐션도 조정됨



- (a) 두 문장을 입력 받아 둘 간의 관계 분류
(b) 한 문장을 입력 받아 일반적인 문서 분류 문제

14.4 사전학습된 BERT 모형의 직접 사용방법

- BERT는 사전학습된 모형을 전이학습으로 활용할 수 있다는 점이 가장 매력적임
- 트랜스포머나 BERT, GPT 모두 막대한 데이터와 학습을 위한 엄청난 자원 및 시간을 요구해 개인이 직접 처음부터 학습해서 사용하는 것은 거의 불가능에 가까움

14.4 사전학습된 BERT 모형의 직접 사용방법

- 공개된 모형을 이용해서 원하는 작업을 수행해 보자
 - “sentiment-analysis” : 감성분석
 - “text-classification” : 문서분류
 - “question-answering” : 질의 응답
 - “text-generation” : 문서 생성
 - “translation” : 기계번역
 - “summarization” : 문서요약

14.4 사전학습된 BERT 모형의 직접 사용방법

▪ “sentiment-analysis” : 감성분석

```
[2]: ! pip install transformers
```

```
[7]: from transformers import pipeline
```

```
> clf = pipeline("sentiment-analysis")  
result = clf("what a beautiful day!")[0]
```

No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revision 714eb0f (<https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>).
Using a pipeline without specifying a model name and revision in production is not recommended.

config.json: 100% 629/629 [00:00<00:00, 72.7kB/s]

model.safetensors: 100% 268M/268M [00:07<00:00, 39.1MB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 6.13kB/s]

vocab.txt: 232k/? [00:00<00:00, 15.8MB/s]

Device set to use cpu

[+ Code](#) [+ Markdown](#)

```
[9]: print("감성분석 결과: %s, 감성스코어: %0.4f" % (result['label'], result['score']))
```

감성분석 결과: POSITIVE, 감성스코어: 0.9999

14.4 사전학습된 BERT 모형의 직접 사용방법

▪ “text-generation” : 문서 생성

```
[2]: ! pip install transformers
```

```
[5]: from transformers import pipeline

text_generator = pipeline("text-generation")
result = text_generator("Alice was beginning to get very tired of sitting by her sisyer on the bank,")
```

No model was supplied, defaulted to openai-community/gpt2 and revision 607a30d (<https://huggingface.co/openai-community/gpt2>).
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use cpu
Setting `pad\_token\_id` to `eos\_token\_id`:50256 for open-end generation.

```
print(result[0]['generated_text'])
```

Alice was beginning to get very tired of sitting by her sisyer on the bank, and, after a few minutes' reading, she began to smile, and told me that she was glad to hear that she had been able to get a nice piece of paper from the bank. "A good deal of money," she said. "You are so thankful for the great paper you have, and I would like to thank you for letting me get a good piece of paper from your bank!" "It is very good paper, Mr. Morgan," I replied. "I am quite sure it will be worth what you paid for it, and I am glad it does not cost some trouble to get it. I am pleased to see you are now tal king of the matter with the bankers at your house, and I am very glad to see you are using the money you give me. I think it is very very nice. I will give you something to say. I have not spoken to th e bankers since I met you, and I do not know how you have come to hear of my being able to get a good piece of paper from the bank, but I hope you will give me a good word of thanks for the great paper you have given me." I made her laugh, and told her that she was sorry he had asked for the paper. She gave him

+ Code + Markdown

14.5 자동클래스를 이용한 토큰나이저와 모형의 사용

- 자동 클래스를 사용해서 자동으로 적절한 토큰나이저와 모형을 선택해볼 수 있음
- 사전학습 모형 “Bert-base-cased-finetuned-mrpc”
 - 사전학습 모형을 가지고 다시 mrpc (The Microsoft Research Paraphrase Corpus)로 미세조정학습을 했음을 의미함
- 사용가능한 사전학습 모델
 - bert-base-uncased
 - distillbert-base-uncased

Bert : 원 모델

Cased : 대소문자 차이를 그대로 유지

Uncased : 대소문자 구분 없이 소문자로 통일

Distillbert : knowledge distillation 방식으로 압축해서 만든 경량화 모델을 의미함

14.5 자동클래스를 이용한 토크나이저와 모형의 사용

- 두 문장의 유사도 분류하기 (유사함/유사하지 않음)

태스크 유형	예시 모델	tokenizer 입력 개수
단일 문장 분류	SST-2 감정 분석	1개
문장 쌍 분류	MRPC, QQP	2개
NLI	MNLI, SNLI	2개
텍스트 생성	GPT, LLaMA	보통 1개
QA	SQuAD	질문 1개 + 문맥 1개 (총 2개)

- "pt" → PyTorch 텐서(torch.tensor)
 - "tf" → TensorFlow 텐서(tf.Tensor)
 - "np" → NumPy 배열(np.ndarray)
- 으로 반환하라는 옵션.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
```

```
# Auto Classes를 이용해 사전학습된 내용에 맞는 토크나이저와 모형을 자동으로 설정
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased-finetuned-mrpc")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-cased-finetuned-mrpc")
```

```
# 의미적으로 유사한 두 문장을 선언
```

```
input_sentence = "She angered me with her inappropriate comments, rumor-spreading, and disrespectfulness at the formal dinner table"
target_sequence = "She made me angry when she was rude at dinner"
```

```
# 토크화
```

```
tokens = tokenizer(input_sentence, target_sequence, return_tensors="pt")
```

```
# 모형으로 결과를 예측 - 소프트맥스 레이어 전의 신경망 결과
```

```
logits = model(**tokens).logits
```

```
# 소프트맥스를 이용해 결과값을 클래스에 대한 확률로 변환
```

```
results = torch.softmax(logits, dim=1).tolist()[0]
```

```
for i, label in enumerate(['no', 'yes']):
```

```
    print(f"{label}: {int(round(results[i] * 100))}%")
```

```
model.to(torch.device('cpu'))
```

```
no: 29%
yes: 71%
```

14.5 자동클래스를 이용한 토큰라이저와 모형의 사용

■ NLTK 영화 리뷰 데이터의 감성 분석 하기 (긍정/부정)

태스크 유형	예시 모델	tokenizer 입력 개수
단일 문장 분류	SST-2 감성 분석	1개
문장 쌍 분류	MRPC, QQP	2개
NLI	MNLI, SNLI	2개
텍스트 생성	GPT, LLaMA	보통 1개
QA	SQuAD	질문 1개 + 문맥 1개 (총 2개)

- "pt" → PyTorch 텐서(torch.tensor)
 - "tf" → TensorFlow 텐서(tf.Tensor)
 - "np" → NumPy 배열(np.ndarray)
- 으로 반환하라는 옵션.

```
[1]: import nltk
      from nltk.corpus import movie_reviews
      from sklearn.model_selection import train_test_split #sklearn에서 제공하는 split 함수를 사용
      import numpy as np

      nltk.download('movie_reviews')
      fileids = movie_reviews.fileids() #movie review data에서 file id를 가져옴
      reviews = [movie_reviews.raw(fileid) for fileid in fileids] #file id를 이용해 raw text file을 가져옴
      categories = [movie_reviews.categories(fileid)[0] for fileid in fileids]

      # label을 0, 1의 값으로 변환
      label_dict = {'pos':1, 'neg':0}
      y = np.array([label_dict[c] for c in categories])

      X_train, X_test, y_train, y_test = train_test_split(reviews, y, test_size=0.2, random_state=7)
      print('Train set count: ', len(X_train))
      print('Test set count: ', len(X_test))

      [nltk_data] Downloading package movie_reviews to
      [nltk_data] /usr/share/nltk_data...
      [nltk_data] Package movie_reviews is already up-to-date!
      Train set count: 1600
      Test set count: 400
```

14.5 자동클래스를 이용한 토큰라이저와 모형의 사용

■ NLTK 영화 리뷰 데이터의 감성 분석 하기 (긍정/부정)

```
[2]: from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
import torch.nn.functional as F

# cuda를 이용한 GPU 연산이 가능하면 cuda를 사용하고, 아니면 cpu를 사용
device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')

# Auto Classes를 이용해 사전학습된 내용에 맞는 토큰라이저와 모형을 자동으로 설정
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased-finetuned-sst-2-english")

# 모델을 gpu로 옮겨서 연산을 준비
model = model.to(device)

batch_size = 10 # 모형으로 한번에 예측할 데이터의 수
y_pred = [] # 전체 예측결과를 저장

num_batch = len(y_test)//batch_size

for i in range(num_batch):
    inputs = tokenizer(X_test[i*batch_size:(i+1)*batch_size], truncation=True, padding=True, return_tensors="pt")
    # 토큰화 결과를 GPU로 이동
    inputs = inputs.to(device)
    # 모형으로 결과를 예측
    logits = model(**inputs).logits
    # 결과값을 클래스에 대한 확률로 변환
    pred = F.softmax(logits, dim=-1)
    # 예측결과를 CPU로 가져와서 넘파이로 변환한 후, argmax로 확률이 가장 큰 클래스를 선택함
    results = pred.cpu().detach().numpy().argmax(axis=1)

    # 전체 예측결과에 추가
    y_pred.extend(results.tolist())

# gpu 메모리를 비움
torch.cuda.empty_cache()

score = sum(y_test == np.array(y_pred))/len(y_test)
print("NLTK 영화리뷰 감성분석 정확도:", score)
```

SST-1 : Stanford sentiment Treebank 5class
SST-2 : Stanford sentiment Treebank 2 class

NLTK 영화리뷰 감성분석 정확도: 0.8425